

This study guide covers the topics and skills you will need for Exam 1. The exam has four questions covering material from four lecture sets. You will have 120 minutes to complete the exam. Focus on understanding concepts well enough to apply them, not just memorize definitions.

Exam Format

- Question 1: Web Application Fundamentals (25 pts) - diagram drawing, RESTful API design, and explaining MVC in context
- Question 2: Story Flows (25 pts) - writing a complete story flow from a scenario description
- Question 3: Controllers (20 pts) - multiple choice questions on ASP.NET MVC controllers and routing
- Question 4: Models, CRRUD, and Code First (30 pts) - mapping a story flow to the architecture and writing controller code

Topic 1: Web Application Development

Lecture: 1-WebAppDevelopment

What You Should Know:

- The client-server architecture: what role the client plays, what role the server plays, and how they communicate
- What intermediate layers exist between client and server (proxies, caches, CDNs, load balancers) and what purpose they serve
- HTTP and HTTPS: the request-response paradigm, what it means for HTTP to be stateless, and how cookies enable stateful sessions
- The difference between HTTP methods: GET, POST, PUT, DELETE - and when each is used
- The structure of an HTTP request and an HTTP response (start line, headers, body)
- REST: what the acronym stands for and all six guiding constraints, especially statelessness and uniform interface
- What a resource is in REST and how resources are identified (URIs)
- URL structure: scheme, host, port, path, query string, anchor
- The MVC architectural pattern: Model, View, Controller - what each is responsible for and how they interact
- Advantages and disadvantages of MVC

What You Should Be Able to Do:

- Draw and label a client-server architecture diagram showing clients, servers, intermediate layers, and protocols
- Design RESTful API endpoints: choose the correct HTTP method, write a proper URI, explain statelessness, and select appropriate response status codes
- Apply the MVC pattern to a specific scenario: describe what the Model, View, and Controller each do and how they interact with each other

Topic 2: Writing Story Flows

Lecture: StoryFlowsLectureNotes

What You Should Know:

- The five core components of a story flow: user story statement, trigger, flow steps, guards/exception flows, and alternative flows
- The correct format for a user story statement: "As an application user, I want to [action] so that [benefit/goal]"
- What a trigger is and how to specify one (HTTP method + URL pattern)
- What guards are and when they are used (validation checks, existence checks, authentication)
- The difference between an exception flow (what happens when a guard fails) and an alternative flow (different paths based on user choices or system states)
- The common CRUD patterns for Create, Read, Update, and Delete operations
- How to write in active voice from the user's perspective

What You Should Be Able to Do:

- Write a complete story flow from a scenario description that you have not seen before
- Use proper formatting conventions: numbered main steps, sub-numbered exception and alternative flows (e.g., 5.1, 5.1.1)
- Include appropriate guard conditions that check for things like data validity and resource existence
- Write alternative flows that handle different user decisions or system states
- Use "Exit flow" correctly to terminate exception paths
- Ensure all paths in your flow converge or exit cleanly

Study Tip: Practice by writing story flows for features you haven't seen in the examples (e.g., user registration, password reset, adding items to a shopping cart). Review the book *CRUD examples* in the lecture notes and make sure you understand the pattern well enough to apply it to a new domain.

Topic 3: Controllers in ASP.NET MVC

Lecture: 1-Controllers

What You Should Know:

- Controller classes: naming convention (must end with "Controller"), inheritance from Controller, and how URLs map to controllers
- Action methods: what they are (public methods of controller classes), what they return (ActionResult), and how they handle GET vs. POST requests

- Routing: conventional routing vs. attribute routing, the default route pattern `{controller=Home}/{action=Index}/{id?}`, and what each segment means
- Route parameters: required vs. optional (the `?` suffix), default values (e.g., `controller=Home`), and how parameter names in patterns must match method parameter names
- Custom routes: how to use literal segments, how to specify controller and action with the defaults anonymous object, and route ordering (more specific routes first)
- Variable-length URL segments using the asterisk `(*)` catch-all parameter
- The `[Bind(Prefix = ...)]` attribute for renaming parameters
- Action result types and their factory methods: `View()`, `RedirectToAction()`, `Content()`, `Json()`, `File()`
- How to access `appsettings.json` values via `IConfiguration` injection
- The `[HttpPost]` attribute and how forms submit data via POST
- How the logger is injected and used in a controller

What You Should Be Able to Do:

- Determine which controller class and action method a given URL will route to, based on a route pattern
- Identify correct HTTP methods, attributes, and return types for action methods
- Read a route pattern and understand what URLs it will match and what parameter values will be extracted
- Choose the correct factory method for a given response type (returning a view, redirecting, returning JSON, etc.)

Study Tip: Go through the code examples in the lecture notes and make sure you can trace a URL to the correct controller and action method. Pay attention to the details in the route patterns - default values, optional parameters, and how custom routes override the default.

Topic 4: Models, CRRUD, and Code First

Lecture: CSCI3110-CRRUD-CodeFirst

What You Should Know:

- What a Model is: an abstraction of objects from the problem domain, and how it maps to a C# entity class
- The ASP.NET MVC dependency structure: the Model does not depend on the View or Controller, but both depend on the Model
- What an ORM is and what Entity Framework does
- The three EF approaches: Code First, Model First, Database First - and which one we use in class
- The course's class pattern: Controller → Repository Interface → Repository Implementation → DbContext → Database

- Entities, the DbContext class, DbSet properties, and how they relate to database tables
- Connection strings: where they are stored (appsettings.json) and how they are linked to the DbContext in Program.cs
- EF Core migrations: what they do, the commands to add a migration and update the database
- The CRRUD operations: Create, Read, Read All, Update, Delete - and the repository pattern for each
- Asynchronous programming: async/await, I/O-bound vs. CPU-bound, and why we use async for database operations
- Service lifetimes: Transient, Scoped, and Singleton - and which is used for repositories (Scoped)
- Dependency injection: how the repository is injected into the controller and how the DbContext is injected into the repository
- ModelState.IsValid: what it checks and what happens when validation fails

What You Should Be Able to Do:

- Map steps in a story flow to the correct architectural components (View, Controller, Repository, DbContext/Database) and explain what each layer does at that step
- Write controller action methods for CRRUD operations, given an entity and a repository interface
- Write a GET action method that returns a view (for displaying a form)
- Write a POST action method that validates input (ModelState.IsValid), calls the repository, and either redirects on success or returns the view on failure
- Use the correct return types, attributes ([HttpPost]), async/await patterns, and RedirectToAction calls
- Understand the flow of data from user input through the View → Controller → Repository → Database and back

Study Tip: Study the PersonController examples in the CRRUD lecture notes closely. Make sure you can write a Create or Update controller action from memory, following the pattern of GET (display form) and POST (process form with validation). Practice tracing a story flow step by step and identifying which layer of the architecture is doing the work at each step.

General Exam Tips

1. Read each question carefully before you start writing. Pay attention to exactly what is being asked.
2. Manage your time. The suggested times for each question add up to 80 minutes, leaving you 40 minutes of buffer. Don't spend too long on any single question.
3. For diagram questions, label everything clearly. A simple, well-labeled diagram is better than a complex, unclear one.

4. For the story flow question, follow the formatting conventions from the lecture notes exactly. Use numbered steps, sub-numbering for exception and alternative flows, and "Exit flow" statements.
5. For the coding question, focus on getting the structure right: correct method signatures, [HttpPost] attribute, ModelState.IsValid check, repository calls with await, and correct return statements (RedirectToAction vs. View).
6. Review all four sets of lecture notes, paying special attention to the code examples and architectural diagrams.